BITE303P
Operating System
Fall Semester 2026-27

ProcNanny — Automatic Process Priority Adjuster Based on CPU Usage

Name:
Dhanush R, 24BIT0178
Preetham H, 24BIT0294
Amit Raj, 24BIT0621

REVIEW DOCUMENT

ProcNanny — Automatic Process Priority Adjuster Based on CPU Usage

Critical review of the Operating Systems Lab (BITE303P) project, Fall 2026-27

1. Abstract

This document presents a critical review of the ProcNanny project, developed for the Operating Systems Lab (BITE303P). ProcNanny is a user-space Linux daemon that samples per-process CPU usage with psutil, lowers the priority (raises the nice value) of processes that stay above a CPU threshold for a sustained period, and gradually restores priority through an aging rule to avoid starvation. Every decision is logged. The review assesses the project's concept, design, technical soundness, experimental evidence, literature survey, references and documentation.

The project has a clear motivation, a well-organised six-module design, a useful comparison of related work and a sensible, conservative aging idea. However, three problems limit its quality in its present form. First, the results are illustrative sample values, yet the summary claims are worded as achieved outcomes. Second, the priority-restoration step may not work without elevated privileges, which conflicts with the stated design. Third, several design details (step size, CPU normalisation, escalation control, process exclusion) are unspecified. Documentation gaps such as an empty table of contents and placeholder student details also remain. With real measurements and the fixes listed in Section 9, the project can become a strong submission.

Overall verdict: Good concept and structure; needs real experimental data and several technical corrections before final submission. Indicative rating: 6.5 / 10 now, achievable 8.5 / 10 after revision.

2. Project Background and Working Principle

2.1 The problem being addressed

Linux schedules ordinary processes with the Completely Fair Scheduler (CFS), which divides CPU time according to each process's nice value (–20 to +19). That value is fixed when the process starts, usually at 0, and changes only if a person runs nice or renice. On a shared workstation, build server or lab machine, a single long-running job such as a compilation, video render, training run or backup can therefore occupy a large share of the CPU for minutes or hours, while interactive processes receive a smaller and less predictable share. Nothing in the default system notices this and corrects it, and manual re-nicing does not scale.

2.2 How ProcNanny works

ProcNanny acts as a "nanny" for the process table. A single background daemon repeats the following loop at a fixed polling interval:

- **Monitor:** read the PID, name, CPU percentage and current nice value of every visible process through psutil and the /proc filesystem.
- **Remember:** keep a short rolling history of CPU samples per process, so brief spikes are smoothed out and only sustained behaviour matters.

- **Decide:** if the average CPU usage stays above the high threshold for the minimum duration, flag the process for de-prioritisation; if a previously de-prioritised process stays below the low threshold for the aging interval, flag it for one-step restoration.
- **Act:** apply the new nice value with os.setpriority() (the same mechanism as renice), never going beyond the allowed bounds.
- **Log:** append a record with timestamp, PID, process name, old and new nice value and a reason code (cpu_hog_detected or aging_restore).

Because the next round of samples reflects the process's behaviour under its new priority, the loop is a feedback system. All thresholds, intervals and step sizes are read from a YAML configuration file, so the tool can be tuned without changing code.

2.3 Scope and key parameters

ProcNanny works entirely in user space on Linux (developed on Ubuntu 22.04 LTS). It does not modify the kernel, introduce a new scheduling policy such as SCHED_FIFO or SCHED_RR, or manage system (root/kernel) processes. It is a policy layer on top of the nice-value mechanism that Linux already provides.

3. Strengths

- **Clear, practical motivation.** The static-nice-value limitation of CFS is a real and easily understood problem, and the "nanny" analogy is memorable.
- **Well-defined scope.** The project states what it will and will not do (no kernel changes, no new scheduling policy, no system processes), which prevents over-claiming.
- **Measurable objectives.** The objectives map directly onto modules and onto the evaluation metrics, so the work is easy to check against its own goals.
- **Clean modular architecture.** The separation of monitoring, history, decision, action and logging, with an explicit feedback loop, is easy to follow, implement and extend.
- **Thoughtful safeguards.** The rolling window filters short spikes, aging is applied one step at a time to avoid oscillation, and the daemon aims never to lower nice below the original value.
- **Good comparison of related work.** Six works are compared by technique, platform, advantages and limitations, which leads naturally to a clearly stated research gap.
- **Auditability.** Structured logging with reason codes makes the system verifiable and provides raw data for evaluation.
- **Sensible metrics.** Jain's fairness index, waiting time, adjustment frequency and daemon overhead together give a balanced evaluation plan.
- **Realistic future work.** cgroups v2, adaptive thresholds, a dashboard, systemd packaging and memory/I/O awareness are credible next steps.

4. Technical Review

4.1 Privilege model of the aging rule

The design assumes that raising nice needs no privilege and that restoring priority only moves “back toward, never below” the original value. On Linux, an unprivileged process cannot decrease its nice value at all (without a suitable RLIMIT_NICE or CAP_SYS_NICE), even when returning to a value it previously held. The aging_restore action will therefore fail with a permission error unless the daemon runs as root or is granted CAP_SYS_NICE. The project should either (a) state that the daemon requires elevated privileges and discuss the security implications, or (b) grant CAP_SYS_NICE or set RLIMIT_NICE, and explain how this was done during testing.

4.2 Escalation and cooldown in the control loop

In the control-loop pseudocode, the de-prioritisation branch has no cooldown. If a process stays above the threshold, nice is increased by STEP on every polling cycle, so it may reach MAX_NICE within a few polls rather than gradually. The aging branch has a cooldown (last_boost), but de-prioritisation has no equivalent. A last_demote timestamp, or clearing the history window after each adjustment, would prevent runaway escalation and make the feedback loop more stable.

4.3 Inconsistency between description and pseudocode

The decision logic is described using a “wait score” for low-usage and de-prioritised processes, but the pseudocode uses only last_boost and an avg < CPU_LOW_THRESHOLD test. The helper sustained_for() is used but never defined, and the default STEP value is never stated. The description and the algorithm should express exactly the same logic, and all defaults should appear in one parameter table.

4.4 CPU percentage semantics

psutil’s cpu_percent() is relative to a single core and can exceed 100% for multi-threaded processes on a multi-core machine (the test VM has 4 vCPUs). A 70% threshold therefore means different things for single- and multi-threaded processes. The project should state whether values are normalised by the core count. Also, the first cpu_percent() call for each process returns 0.0 and must be “primed”; otherwise the first history sample is misleading.

4.5 Effect size of nice adjustments

Under CFS, each nice step changes a task’s weight by roughly 25%. A shift in CPU share from about 40% to about 23% would require several steps per process within a 5-minute window, given a 30 s sustained-duration requirement and 5 s polling. The actual nice trajectory of each process (from procnanny.log) should be shown to prove that the change in share is caused by ProcNanny and is achievable with the chosen STEP and bounds.

4.6 Safety and exclusion policy

ProcNanny monitors all user-owned processes. A blanket policy could de-prioritise legitimate foreground work (an IDE, a browser, a video call) or critical user services. The configuration offers no whitelist/blacklist, minimum-priority protection or dry-run mode. Adding an exclusion list and a dry-run (log-only) mode would improve safety and credibility.

4.7 Process trees and workload handling

stress-ng spawns worker processes, and many real jobs (make, browsers) use process trees. It is not stated whether ProcNanny adjusts a parent, each child, or a process group, and short-lived children that disappear between polls are simply dropped. This is acceptable but should be discussed as a limitation, since it affects fairness measurements.

4.8 Kernel context

Describing CFS as the Linux scheduler is accurate for Ubuntu 22.04 (kernel 5.15), but newer kernels (6.6 and later) replaced CFS with EEVDF. A short note on this would make the project more current and show awareness of scheduler evolution.